TZ — TarefaZen

Sistema Colaborativo de Gestão de Tarefas

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	TZ — TarefaZen
Gerente de Projeto	Guilherme Vieira Wesolowski
Gerente/Responsável de GCS/DevOps	Guilherme Vieira Wesolowski
Fábrica de Software / Squad	Individual — Guilherme Vieira Wesolowski (atividade acadêmica)
Repositório principal	https://github.com/guivieira7/tarefazen

Histórico de Revisões

Data	Versão	Descrição	Autor
22/09/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD).	Guilherme Vieira Wesolowski

Sumário

1. Introdução 3

1.1 Finalidade 3
1.2 Escopo 3
1.3 Definições, Acrônimos e Abreviações 3
1.4 Referências 3
1.5 Visão Geral 4

2. Cultura DevOps e Papéis na Gerência de Configuração 4

3. Ferramentas, Ambientes e Infraestrutura 4

3.1 Ferramentas do Pipeline 4
3.2 Ambientes e Infraestrutura 5
3.3 Estratégia de Branching e Versionamento 5

4. Identificação da Configuração 6

4.1 Itens de Configuração 6
4.2 Convenção de Nomenclatura 6
4.3 Estrutura do Repositório 6

5. Integração Contínua (CI) 6

5.1 Etapas do Pipeline de CI 6
5.2 Política de Pull Request e Revisão de Código 7
5.3 Critérios de Qualidade (Quality Gate) 7

6. Entrega e Implantação Contínua (CD) 7

6.1 Etapas do Pipeline de CD 7
6.2 Estratégias de Implantação 7
6.3 Rollback e Recuperação 7

7. Infraestrutura como Código (IaC) 7

7.1 Versionamento de Infraestrutura 7
7.2 Ambientes Efêmeros 8

8. Controle de Configuração e Mudança 8

8.1 Processo de Solicitação de Mudança 8
8.2 Comitê de Controle de Mudança (CCM) 8

9. Baselines e Releases 8

9.1 Geração Automática de Baseline 8
9.2 Notas de Versão (Release Notes) 8

10. Estimativa do Status de Configuração 8

10.1 Monitoramento e Observabilidade 8
10.2 Métricas DORA 8
10.3 Relatórios e Auditorias 9

11. Marcos 9

12. Treinamento e Recursos 9

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros 9

14. Quadro-Resumo do Pipeline CI/CD 9

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto TarefaZen, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido no desenvolvimento do TarefaZen, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGC é destinado ao desenvolvedor individual responsável pela evolução, build, testes e implantação do sistema TarefaZen — uma aplicação web de gestão colaborativa de tarefas, composta por uma API back-end (Node.js) e um front-end (React), com persistência em banco de dados relacional. O escopo abrange todo o código-fonte da aplicação, os scripts de infraestrutura e os pipelines de CI/CD do projeto.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) — ex.: 1.0.0
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.
- Repositório do projeto TarefaZen (README e ADRs).
- Cronograma acadêmico da disciplina.

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação. Como o TarefaZen é desenvolvido individualmente, todos os papéis abaixo são exercidos pelo mesmo desenvolvedor, alternando de função conforme a atividade.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Prioriza o backlog, autoriza mudanças de maior impacto e acompanha os indicadores de entrega (DORA Metrics).
Engenheiro DevOps / Release Engineer	Projeta e mantém os pipelines de CI/CD, define a estratégia de branching e versionamento, administra o registro de artefatos/imagens e a IaC.
Desenvolvedor(a)	Integra código com frequência na branch principal, escreve testes automatizados e é responsável pela qualidade do que é implantado.
SRE / Operações	Monitora o ambiente de produção, define indicadores de saúde (health checks) e realiza rollback quando necessário.
QA / Automação de Testes	Mantém a suíte de testes automatizados (unitários e de integração com Vitest) executada nos pipelines de CI e CD.
Segurança (DevSecOps)	Define políticas de análise de dependências (SCA) e gestão de segredos integradas ao pipeline.
Comitê de Controle de Mudanças (CCM)	Não formalizado — dado o projeto individual, mudanças de alto risco são registradas e justificadas no próprio Pull Request antes do merge.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Categoria	Ferramenta(s) adotada(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento de código-fonte, IaC e pipelines como código
Orquestração de CI/CD	GitHub Actions	Automação de build, testes, análise e implantação
Empacotamento	Docker	Geração de imagens versionadas e imutáveis da API e do front-end
Registro de artefatos	GitHub Container Registry (GHCR)	Armazenamento e versionamento das imagens de container

Categoria	Ferramenta(s) adotada(s)	Finalidade
Infraestrutura como Código	Terraform	Provisionamento e configuração versionada dos ambientes de staging e produção
Orquestração de containers	Docker Compose (staging/produção em VPS única)	Execução dos serviços implantados (API, front-end, banco de dados)
Gestão de segredos	GitHub Actions Secrets	Armazenamento seguro de credenciais usadas nos pipelines
Monitoramento e Observabilidade	Prometheus + Grafana	Métricas, logs e alertas dos ambientes
Gestão de dependências / SCA	Dependabot	Identificação de vulnerabilidades em bibliotecas de terceiros
Rastreamento de tarefas	GitHub Issues + GitHub Projects	Vinculação de commits/PRs a itens de trabalho

3.2 Ambientes e Infraestrutura

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Execução local do desenvolvedor; recriado a cada execução do docker-compose local.	Manual, a cada ciclo de desenvolvimento
Homologação / Staging	Testes de aceitação, integração entre API e front-end, validação funcional antes da produção.	Pipeline de CI aprovado no merge para a branch main
Produção	Ambiente utilizado pelos usuários finais do TarefaZen.	Aprovação manual do gate de release após validação em staging

Os ambientes de Staging e Produção são provisionados em uma VPS via Terraform (módulo de infraestrutura versionado em /infra), com topologia composta por: API (container Node.js), front-end (container servido via Nginx), banco de dados PostgreSQL (container com volume persistente) e proxy reverso Nginx com certificado TLS. As credenciais de acesso ficam armazenadas nos GitHub Actions Secrets, nunca em texto plano no repositório.

3.3 Estratégia de Branching e Versionamento

Adota-se Trunk-Based Development (GitHub Flow), com branches de vida curta, para viabilizar integração contínua real:

- main — sempre implantável (deployable); protegida por regra de PR obrigatório e pipeline verde.
- feature/<descrição-curta> — branches de curta duração (idealmente < 1–2 dias) para cada mudança.
- hotfix/<descrição> — correções emergenciais aplicadas diretamente sobre produção e retrointegradas à main.

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor...), permitindo geração automática de changelog e da próxima versão pelo pipeline.

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, os seguintes artefatos também são itens de configuração formais, versionados no repositório do TarefaZen:

- Código-fonte da API (Node.js) e do front-end (React), e dos testes automatizados (Vitest).
- Definições de pipeline como código (.github/workflows/ci.yml, .github/workflows/cd.yml).
- Dockerfiles (API e front-end) e o arquivo docker-compose.yml.
- Scripts de Infraestrutura como Código (Terraform, em /infra).
- Arquivos de configuração por ambiente (.env.example, feature flags) — exceto chaves e senhas, mantidas nos GitHub Actions Secrets.
- Scripts de migração de banco de dados (/migrations).
- Documentação técnica (README, ADRs — Architecture Decision Records).

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<ticket>-<descrição-curta>	feature/TZ-12-login-oauth
Commit	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(auth): adiciona login via OAuth2
Tag de release	Versionamento Semântico vX.Y.Z	v1.2.0
Imagem de container	<registro>/<projeto>:<versão|sha>	ghcr.io/guivieira7/tarefazen-api:1.2.0

4.3 Estrutura do Repositório

- /src/api — código-fonte da API Node.js
- /src/web — código-fonte do front-end React
- /infra — scripts Terraform de Infraestrutura como Código
- /.github/workflows — pipelines de CI e CD
- /tests — testes unitários e de integração (Vitest)
- /migrations — scripts de migração de banco de dados
- /docs — documentação técnica, ADRs e diagramas

5. Integração Contínua (CI)

Todo Pull Request para a branch main deve disparar automaticamente um pipeline de CI. Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso.

5.1 Etapas do Pipeline de CI

- 1. Checkout do código e restauração de dependências (com cache do npm).
- 2. Build da API e do front-end.
- 3. Análise estática de código (ESLint) e verificação de padrões de codificação.
- 4. Testes unitários com Vitest, com medição de cobertura mínima.
- 5. Testes de integração da API.
- 6. Análise de segurança de dependências (SCA) via Dependabot/npm audit.

- 7. Empacotamento e publicação das imagens Docker versionadas no GHCR.

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige, no mínimo, autorrevisão documentada (checklist de PR), já que o projeto é individual; quando possível, revisão por colega da disciplina.
- PRs devem ser pequenos e de vida curta, para reduzir conflitos e facilitar revisão.
- Merge somente permitido com pipeline de CI verde (quality gate).

5.3 Critérios de Qualidade (Quality Gate)

- Cobertura de testes automatizados ≥ 70%.
- Zero vulnerabilidades críticas ou altas reportadas pela análise de dependências.
- Zero erros de lint bloqueante (ESLint configurado como etapa obrigatória).

6. Entrega e Implantação Contínua (CD)

Após a publicação das imagens pelo pipeline de CI, o pipeline de CD é responsável por promover essa mesma versão, sem recompilação, através dos ambientes até a produção.

6.1 Etapas do Pipeline de CD

Nota de status: o workflow de CD abaixo já está desenhado e versionado no repositório (.github/workflows/cd.yml), com as etapas de deploy em staging e produção implementadas como simulação (placeholders). A execução real do deploy depende do provisionamento da VPS e da configuração das credenciais de acesso (item ainda pendente nesta fase acadêmica do projeto).

- 1. Deploy automático em ambiente de Homologação/Staging.
- 2. Execução de testes automatizados de aceitação em staging.
- 3. Gate de aprovação manual para produção.
- 4. Deploy em produção com estratégia Rolling Update (ver 6.2).
- 5. Testes de fumaça (smoke tests) pós-deploy e verificação de saúde (health checks).
- 6. Geração automática de baseline/tag de release e notas de versão (changelog).

6.2 Estratégias de Implantação

Estratégia	Descrição
Rolling update	Estratégia adotada: substituição gradual dos containers antigos pelos novos, minimizando indisponibilidade, via docker-compose.
Feature Flags	Utilizadas para funcionalidades experimentais, implantadas desligadas e ativadas em produção de forma independente do deploy.

6.3 Rollback e Recuperação

Toda implantação é reversível. O pipeline de CD mantém a versão anterior disponível (imagem no GHCR, IaC versionada) para permitir rollback manual em caso de falha nos testes de fumaça ou nos indicadores de saúde pós-deploy, bastando reimplantar a tag de imagem anterior.

7. Infraestrutura como Código (IaC)

Toda a infraestrutura provisionada para os ambientes de Staging e Produção é descrita como código versionado em /infra, submetida ao mesmo fluxo de Pull Request e pipeline usado para a aplicação.

Nota de status: o arquivo Terraform em /infra descreve a estrutura de provisionamento planejada, mas ainda não foi aplicado (terraform apply) contra um provedor de nuvem real. Nesta fase, os ambientes de Staging e Produção descritos neste plano são executados via Docker Compose em uma VPS configurada manualmente; a automação completa via IaC é o próximo passo evolutivo do projeto.

7.1 Versionamento de Infraestrutura

- Alterações de infraestrutura seguem o mesmo processo de PR + revisão + pipeline ("plan" automático em PR, "apply" após aprovação).
- O estado da infraestrutura (state file) é armazenado de forma centralizada (backend remoto do Terraform), nunca na máquina local do desenvolvedor.

7.2 Ambientes Efêmeros

Dada a natureza acadêmica e o porte do projeto, não são provisionados ambientes de revisão (review apps) automáticos por Pull Request neste momento; o ambiente de Staging único cumpre esse papel de validação isolada antes da produção.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo padrão: abertura de item de trabalho no GitHub Issues → branch de feature → Pull Request → autorrevisão/revisão → pipeline de CI → merge → pipeline de CD. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança.

8.2 Comitê de Controle de Mudança (CCM)

Por se tratar de um projeto individual, não há CCM formal. Mudanças de alto impacto ou risco elevado — como alterações estruturais de infraestrutura ou mudanças de esquema de banco de dados em produção — são planejadas previamente em um ADR (Architecture Decision Record) registrado em /docs antes da implementação, substituindo a aprovação de comitê por documentação e justificativa explícitas.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Cada implantação bem-sucedida em produção gera automaticamente uma baseline: uma tag de versão (SemVer) associada ao commit, à imagem publicada no GHCR e ao estado da infraestrutura correspondente.

9.2 Notas de Versão (Release Notes)

As notas de release são geradas automaticamente pelo pipeline a partir dos Conventional Commits desde a última tag, publicadas junto à release no repositório do GitHub.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Os ambientes de Staging e Produção são monitorados continuamente via Prometheus e Grafana (métricas de uso de CPU/memória dos containers, tempo de resposta da API e taxa de erros), permitindo identificar rapidamente falhas decorrentes de uma implantação.

Nota de status: Prometheus e Grafana são a stack de observabilidade planejada para este plano; a configuração e o deploy efetivo desses serviços ainda não foram implementados no repositório nesta fase acadêmica, ficando como próximo passo junto à implantação real em produção.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.

10.3 Relatórios e Auditorias

O histórico de pipelines (builds, testes, deploys) do GitHub Actions e os logs do sistema de versionamento constituem o principal insumo de auditoria de GCS, dispensando relatórios manuais na maior parte dos casos.

11. Marcos

- Implantação do pipeline de CI (build, lint, testes, SCA).
- Implantação do pipeline de CD com deploy automatizado em Staging.
- Primeira release automatizada em Produção (v1.0.0).
- Adoção completa de IaC para os ambientes de Staging e Produção.

12. Treinamento e Recursos

Para implementar e manter as práticas de CI/CD, IaC e demais atividades de GCS descritas neste plano, são necessários: domínio de Git/GitHub Flow, familiaridade com GitHub Actions para automação de pipelines, conhecimento básico de Docker e Terraform, e leitura da documentação oficial dessas ferramentas. Como o projeto é individual, o próprio desenvolvedor é responsável por esse autoestudo, apoiado pelo material da disciplina.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas e serviços de terceiros integrados ao TarefaZen também são itens de configuração e são versionados explicitamente via arquivo de dependências travado (package-lock.json).

- Análise automática de vulnerabilidades em dependências (SCA) integrada ao pipeline de CI via Dependabot/npm audit.
- Atualizações de dependências propostas automaticamente pelo Dependabot e submetidas ao mesmo fluxo de PR.

O projeto não envolve subcontratados ou fornecedores externos de desenvolvimento; todas as dependências são bibliotecas open source públicas.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Build, lint, testes unitários/integração, SCA, publicação da imagem.	Abertura/atualização de Pull Request
Merge	Integração da mudança na branch main.	Aprovação de PR + CI verde
CD — Staging	Deploy automático em homologação + testes de aceitação.	Merge na branch main
Gate de Release	Aprovação manual para produção.	Sucesso dos testes em staging
CD — Produção	Deploy (rolling update), smoke tests, geração de baseline/tag.	Aprovação do gate de release

Nota: este plano foi adaptado ao contexto do projeto TarefaZen, mantendo os quatro pilares da GCS — identificação, controle de versão, controle de mudanças e auditoria/status — agora suportados por automação.